Arbre Parfait et Tas

1 Arbre parfait

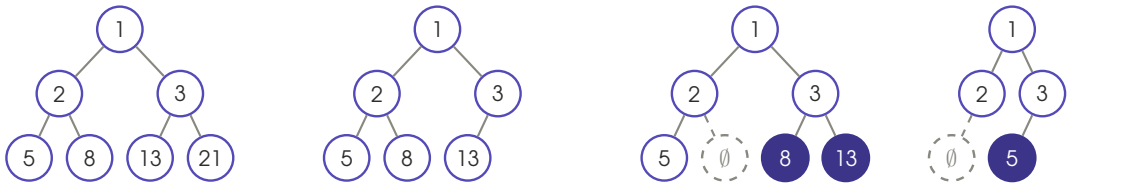
1.1 Définition

Arbre binaire parfait

Un arbre binaire est **parfait** si :

- tous les niveaux sont **complets**, sauf éventuellement le dernier, où il peut manquer des nœuds (feuilles) : l'arbre est alors dit *incomplet* ;
- les feuilles du dernier niveau sont **regroupées à partir de la gauche**.

Exemples et contre-exemples.



Deux arbres parfaits, puis deux arbres qui ne le sont pas (en violet foncé : les feuilles qui posent problème).

Pour retenir. On remplit l'arbre niveau par niveau, de gauche à droite, *sans jamais laisser de trou*. Pour un nombre de nœuds n donné, il n'existe donc **qu'une seule forme** d'arbre parfait.

Propriété

Un arbre parfait à $n \geq 1$ nœuds a pour hauteur $h = \lfloor \log_2 n \rfloor$. Le niveau k contient 2^k nœuds, sauf peut-être le dernier. Ainsi $2^h \leq n \leq 2^{h+1} - 1$. Un arbre parfait est donc **toujours de hauteur minimale**, ce qui garantit des chemins de longueur $\mathcal{O}(\log n)$.

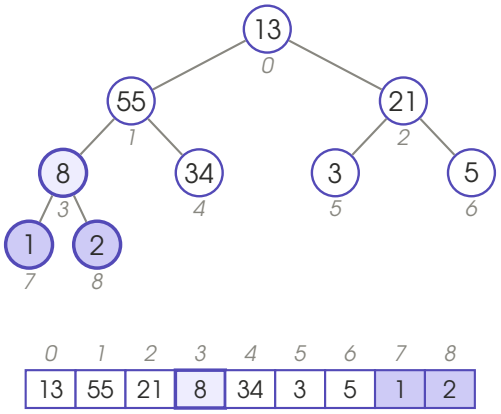
1.2 Implémentation par une liste

Comme il n'y a pas de trou, on range les nœuds dans une liste L, **niveau par niveau et de gauche à droite**. Aucun pointeur n'est nécessaire : la position d'un nœud suffit à retrouver ses voisins.

Règles d'indexation

1. `L[0]` est la racine de l'arbre.
2. Les fils de `L[i]` sont `L[2i+1]` (gauche) et `L[2i+2]` (droit), s'ils existent, c'est-à-dire si leur indice est $< n$.
3. Le père de `L[i]` ($i \geq 1$) est `L[(i-1)//2]`.
4. `L[i]` est une **feuille** si et seulement si $i \geq n//2$.

Exemple.



0	1	2	3	4	5	6	7	8
13	55	21	8	34	3	5	1	2

Les indices sont en gris. Les fils de `L[3]` sont `L[7]` et `L[8]`.

Pourquoi la règle des feuilles ? Le nœud i a un fils gauche si $2i + 1 < n$, soit $i < \frac{n-1}{2}$, ce qui équivaut à $i < n//2$ pour un entier i. Les feuilles sont donc exactement les indices $n//2, \ldots, n-1$: **la moitié environ des nœuds** sont des feuilles.

Vérification sur l'exemple. $n = 9$ et $n//2 = 4$: les nœuds internes ont pour indices $0, 1, 2, 3$ et les feuilles les indices $4, \ldots, 8$, ce que confirme la figure.

2 La structure de tas

2.1 Définition

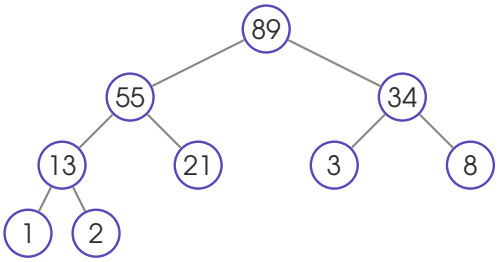
Tas

Un **tas** est un arbre binaire parfait qui vérifie l'une des propriétés suivantes :

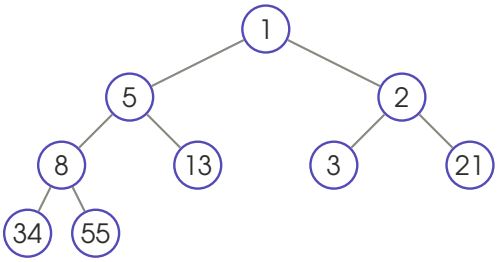
- **tas max** : chaque nœud a une valeur supérieure à celle de ses deux fils ;
- **tas min** : chaque nœud a une valeur inférieure à celle de ses deux fils.

Les valeurs sont supposées **distinctes**.

Exemples.



tas max : chaque père $>$ ses fils



tas min : chaque père $<$ ses fils

Dans la suite, on s'intéresse au **tas max** : sa **racine contient le maximum** des valeurs, auquel on accède en $\mathcal{O}(1)$.

2.2 Réparer un tas : `entasser_max`

Situation

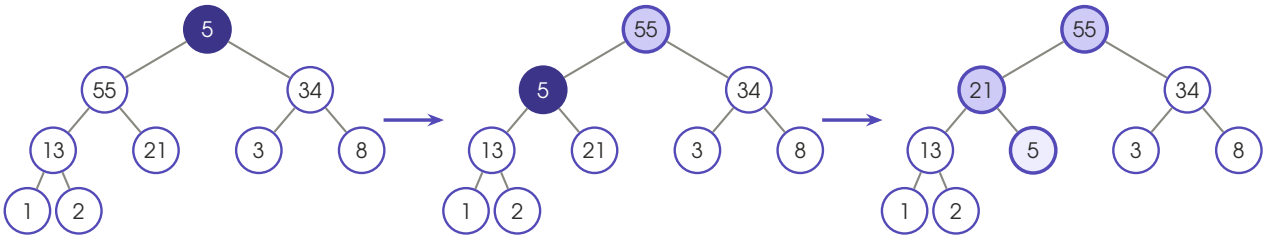
Soit un nœud d'indice i dont les **deux sous-arbres sont déjà des tas max**, mais dont la valeur est peut-être **trop petite** : la propriété n'est éventuellement violée qu'en i.

Idée. On fait *descendre* cette valeur jusqu'à sa place :

1. on compare `L[i]` à ses fils et on repère le plus grand des trois ;
2. si c'est `L[i]`, le tas est réparé ;
3. sinon on échange `L[i]` avec le **plus grand de ses fils**, puis on recommence au même nœud, devenu fils, qui peut encore violer la propriété avec ses nouveaux fils.

Cette opération s'appelle *entasser*.

Exemple.



$i = 0$: 5 est plus petit que ses fils 55 et 34

on échange avec 55, le plus grand fils ; puis $i = 1$: $5 < 13, 21$

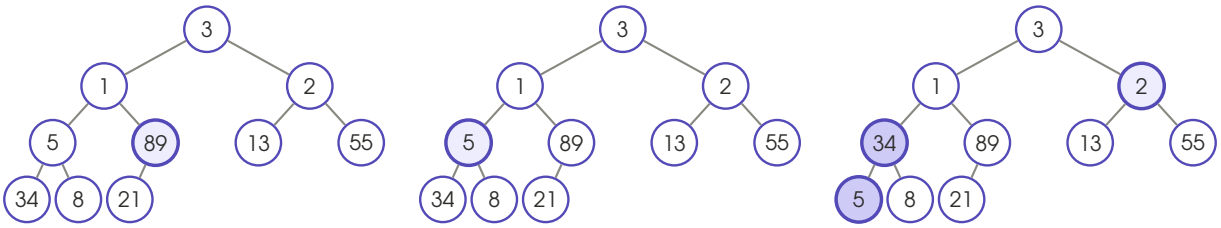
on échange avec 21 ; 5 est une feuille : terminé

Complexité. Chaque appel descend d'un niveau : au plus $h = \lfloor \log_2 n \rfloor$ échanges, donc $\mathcal{O}(\log n)$.

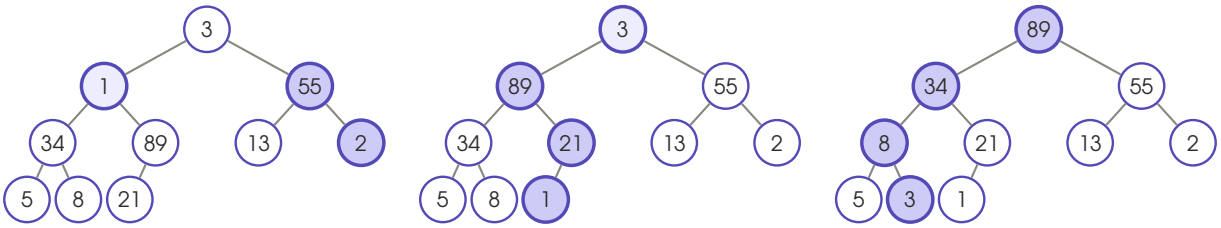
2.3 Construction d'un tas max : `construire_tas`

Les feuilles sont déjà des tas à un seul élément. On parcourt donc les **nœuds internes**, d'indices $n//2 - 1$ **jusqu'à** 0 (des derniers vers la racine), et on appelle `entasser_max` sur chacun. Le sens est essentiel : quand on traite le nœud i, ses deux sous-arbres ont déjà été transformés en tas.

Exemple.



(1) $i = 4$ (2) $i = 3$ (3) $i = 2$



(4) $i = 1$ (5) $i = 0$ (6) tas max obtenu

construire_tas sur `[3, 1, 2, 5, 89, 13, 55, 34, 8, 21]` ($n = 10$, *donc* $i = 4, 3, 2, 1, 0$).

À RETENIR

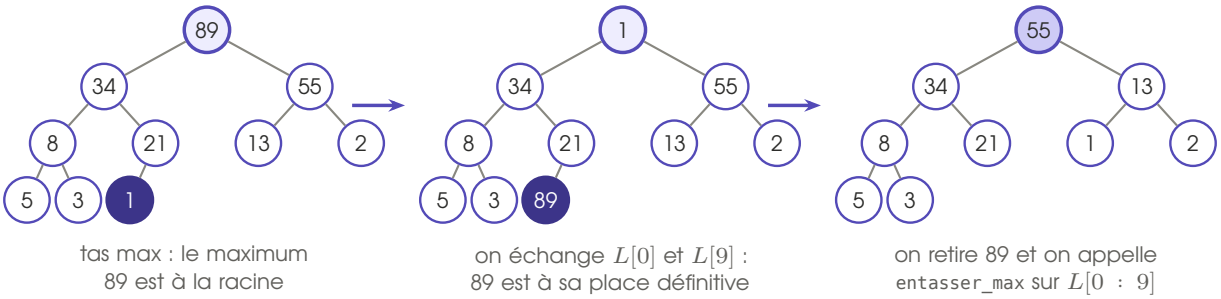
Complexité de `construire_tas` : $\mathcal{O}(n)$ (résultat admis). Un appel sur un nœud de hauteur k coûte $\mathcal{O}(k)$, et il y a peu de nœuds de grande hauteur : environ $n/2$ feuilles ne coûtent rien, $n/4$ nœuds coûtent un échange, $n/8$ en coûtent deux, etc. La somme $\sum_{k=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{k+1}} k$ est en $\mathcal{O}(n)$, et non $\mathcal{O}(n \log n)$ comme une estimation grossière le laisserait croire.

2.4 Tri par tas

Le tri par tas utilise le fait que le **maximum est à la racine** :

1. on construit un tas max sur `L[0:n]` avec `construire_tas` ;
2. on échange la racine `L[0]` avec `L[n-1]` : le maximum est à sa place définitive, on le « retire » du tas (la taille du tas, notée `limite`, diminue de 1) ;
3. les deux sous-arbres de la racine sont encore des tas max, mais la nouvelle racine peut violer la propriété : un seul appel à `entasser_max` restaure le tas sur `L[0:n-1]` ;
4. on répète avec le tas de taille $n - 1$, puis $n - 2$, ... jusqu'à la taille 1.

Exemple.



Première étape du tri sur le tas obtenu à la figure précédente.

Un paramètre en plus. Pendant le tri, le tas rétrécit mais la liste garde sa taille : la fin de `L` contient déjà les éléments triés, et `entasser_max` ne doit pas les considérer comme des fils. On ajoute donc à `entasser_max` un paramètre `limite` : seuls les éléments de `L[0:limite]` font partie du tas (exercice 3). La construction appelle `entasser_max(L, i, len(L))`.

Trace complète. Les cases en violet clair forment la partie triée, qui grandit à droite :

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*
tas max	89	34	55	8	21	13	2	5	3	1
fin $= 9$	55	34	13	8	21	1	2	5	3	89
fin $= 8$	34	21	13	8	3	1	2	5	55	89
fin $= 7$	21	8	13	5	3	1	2	34	55	89
fin $= 6$	13	8	2	5	3	1	21	34	55	89
fin $= 5$	8	5	2	1	3	13	21	34	55	89
fin $= 4$	5	3	2	1	8	13	21	34	55	89
fin $= 3$	3	1	2	5	8	13	21	34	55	89
fin $= 2$	2	1	3	5	8	13	21	34	55	89
fin $= 1$	1	2	3	5	8	13	21	34	55	89

À RETENIR

Complexité du tri par tas : $\mathcal{O}(n \log n)$ **dans tous les cas** : $\mathcal{O}(n)$ pour la construction, puis $n-1$ appels de coût $\mathcal{O}(\log n)$. Le tri se fait **en place** (aucune liste auxiliaire), mais n'est pas stable.

3 Exercices

Exercice 1 — ●○○ Arbres parfaits et tas

1. Écrire les fonctions suivantes, où `L` est la liste qui représente un arbre parfait :
   - `gauche(i)` et `droit(i)` renvoient les indices des fils gauche et droit du nœud d'indice `i` ;
   - `parent(i)` renvoie l'indice du père du nœud d'indice `i` (avec `i >= 1`) ;
   - `est_feuille(L, i)` renvoie `True` si le nœud d'indice `i` est une feuille, et `False` sinon.
2. Écrire une fonction `est_tas_max(L)` qui renvoie `True` si `L` est un tas max, et `False` sinon.

Indication

Pour la question 2, il suffit de vérifier que chaque nœud, sauf la racine, est inférieur à son père : utiliser la formule du père.

Exercice 2 — ●●○ Entasser et construire un tas

1. Écrire une fonction `entasser_max(L, i)` qui fait descendre `L[i]` jusqu'à sa place, en supposant que les deux sous-arbres de i sont déjà des tas max.
2. En déduire une fonction `construire_tas(L)` qui transforme une liste quelconque en tas max.

Indication

Pour la question 1 : calculer les indices `g` et `d` des fils, repérer l'indice `m` du plus grand parmi `i`, `g` et `d` (sans sortir de la liste), puis échanger et recommencer si `m != i`. Pour la question 2 : parcourir les nœuds internes, des derniers vers la racine.

Exercice 3 — ●●○ Tri par tas

1. Ajouter à `entasser_max` un paramètre `limite` : seuls les éléments de `L[0:limite]` font partie du tas. Adapter `construire_tas`.
2. Écrire une fonction `tri_par_tas(L)` qui trie `L` en place par ordre croissant.
3. Que fait `tri_par_tas` sur une liste déjà triée ? Quelle est sa complexité ?
4. Comment modifier les deux fonctions précédentes pour trier par ordre *décroissant* ?

4 Corrigés

À détacher : les codes ci-dessous ont tous été testés.

Exercice 1

Question 1.

```
def gauche(i):
    return 2 * i + 1

def droit(i):
    return 2 * i + 2

def parent(i):
    return (i - 1) // 2            # défini pour i >= 1

def est_feuille(L, i):
    return i >= len(L) // 2
```

Question 2. On compare chaque nœud à son père.

```
def est_tas_max(L):
    for i in range(1, len(L)):
        if L[parent(i)] < L[i]:    # un fils plus grand que son père
            return False
    return True
```

Exercice 2

Question 1.

```
def entasser_max(L, i):
    n = len(L)
    g, d = 2 * i + 1, 2 * i + 2
    m = i                          # indice du plus grand parmi i, g, d
    if g < n and L[g] > L[m]:
        m = g
    if d < n and L[d] > L[m]:
        m = d
    if m != i:                     # le père est trop petit : on le fait descendre
        L[i], L[m] = L[m], L[i]
        entasser_max(L, m)          # le nœud descendu peut violer la propriété plus
    bas
```

Question 2.

```
def construire_tas(L):
    for i in range(len(L) // 2 - 1, -1, -1):   # des derniers nœuds internes jusqu'à la
     racine
        entasser_max(L, i)
```

Exercice 3

Question 1. La seule modification : un fils n'existe que si son indice est inférieur à `limite`. La construction appelle `entasser_max(L, i, len(L))`.

```
def entasser_max(L, i, limite):
    # Hypothèse : les sous-arbres des fils de i (dans L[0:limite]) sont des tas max.
    g, d = 2 * i + 1, 2 * i + 2
    m = i                           # indice du plus grand parmi i, g, d
    if g < limite and L[g] > L[m]:
        m = g
    if d < limite and L[d] > L[m]:
        m = d
    if m != i:                      # le père est trop petit : on le fait descendre
        L[i], L[m] = L[m], L[i]
        entasser_max(L, m, limite)  # le nœud descendu peut violer la propriété plus
    bas

def construire_tas(L):
    for i in range(len(L) // 2 - 1, -1, -1):   # des derniers nœuds internes jusqu'à la
     racine
        entasser_max(L, i, len(L))
```

Question 2.

```
def tri_par_tas(L):
    construire_tas(L)
    for fin in range(len(L) - 1, 0, -1):
        L[0], L[fin] = L[fin], L[0]            # le maximum va à sa place définitive
        entasser_max(L, 0, fin)                # on restaure le tas sur L[0:fin]
```

Question 3. Une liste déjà triée n'est pas un cas favorable : `construire_tas` la bouleverse, puis les $n-1$ extractions coûtent chacune $\mathcal{O}(\log n)$. La complexité reste $\mathcal{O}(n \log n)$, aussi bien dans le meilleur que dans le pire cas (contrairement au tri par ABR, qui devient quadratique).

Question 4. Il suffit de remplacer les comparaisons `>` par `<` : on construit alors un **tas min**, dont la racine est le minimum, et le tri place les petites valeurs à la fin de la liste.